



Dimensional Modeling for Lakehouse

How to get the best star schema performance in Lakehouse



Introduction
Dimensional modeling
Best practices
Support patterns

Agenda

10,000+
global customers

Inventor of the **lakehouse**
&
Pioneer of **generative AI**

\$1.5B+
in revenue



databricks

The data and AI
company

\$4B
in investment

Gartner-recognized Leader
Database Management Systems
Data Science and Machine Learning Platforms

Creator of



DELTA LAKE

mlflow

APACHE
Spark



Unity Catalog

DBRX



Data Lakehouse

An open, unified foundation
for all your data



Generative AI

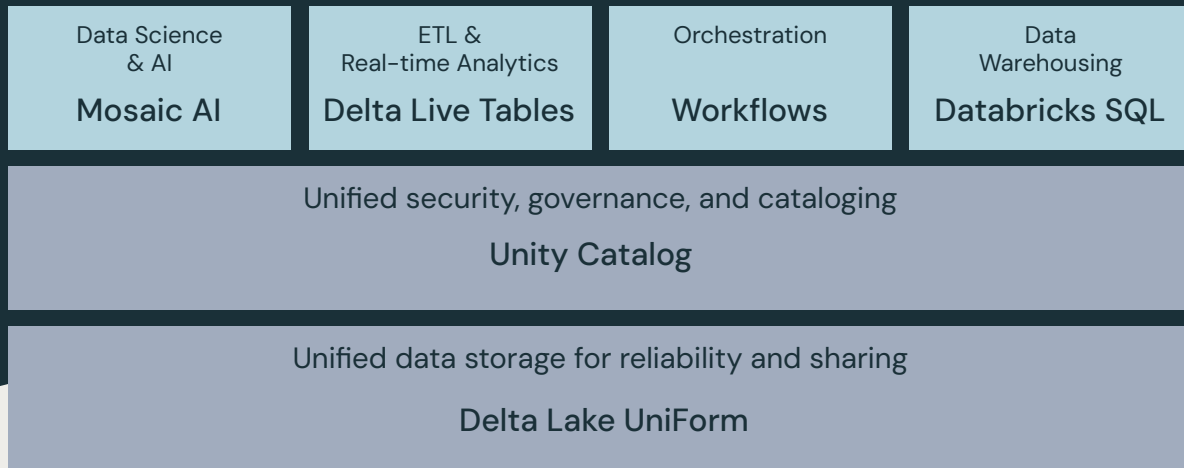
Easily scale and use data and AI



Data Intelligence Platform

Democratize data + AI across
your entire organization

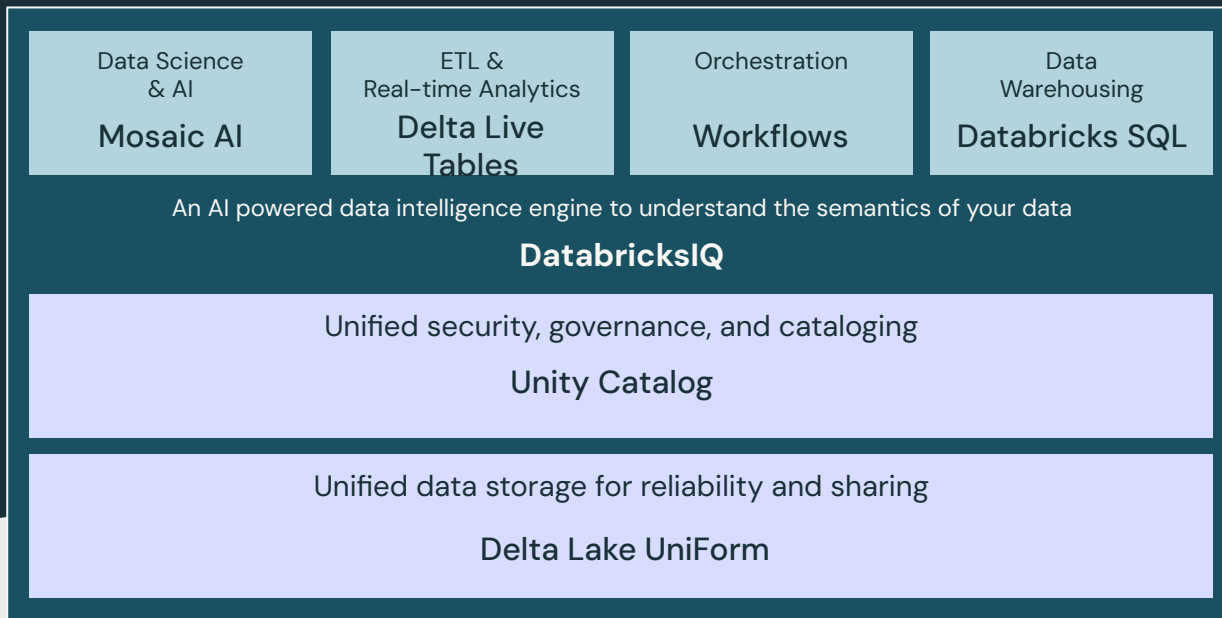
Databricks Data Intelligence Platform



Open Data Lake

All Raw Data
(Logs, Texts, Audio, Video, Images)

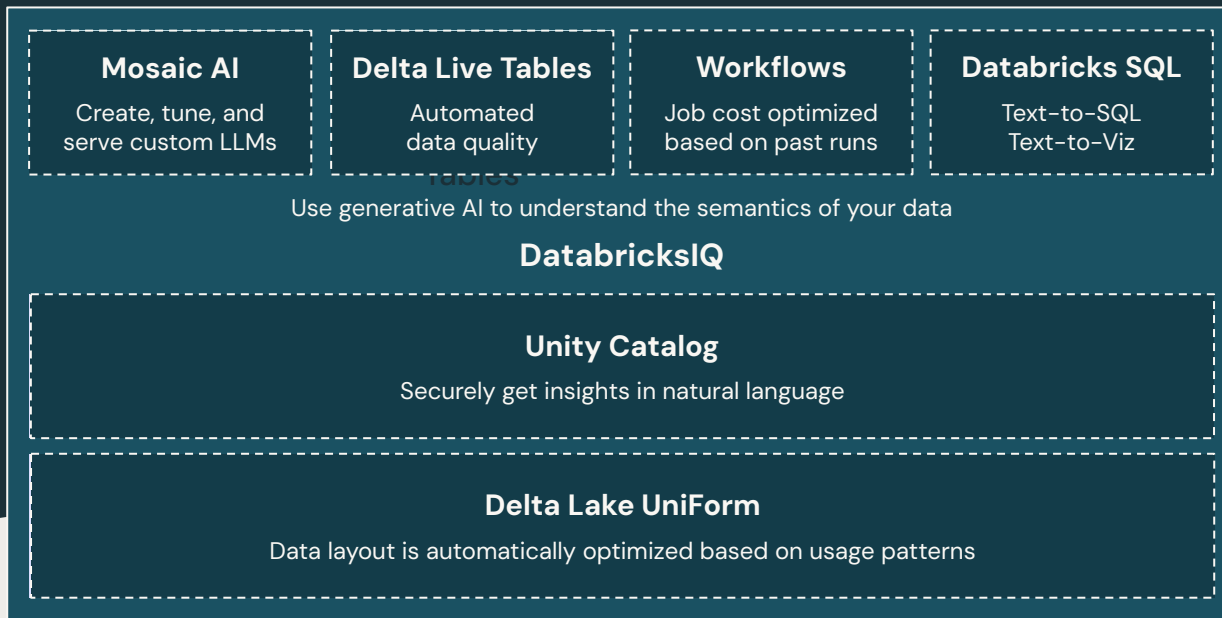
Databricks Data Intelligence Platform



Open Data Lake

All Raw Data
(Logs, Texts, Audio, Video, Images)

Databricks Data Intelligence Platform

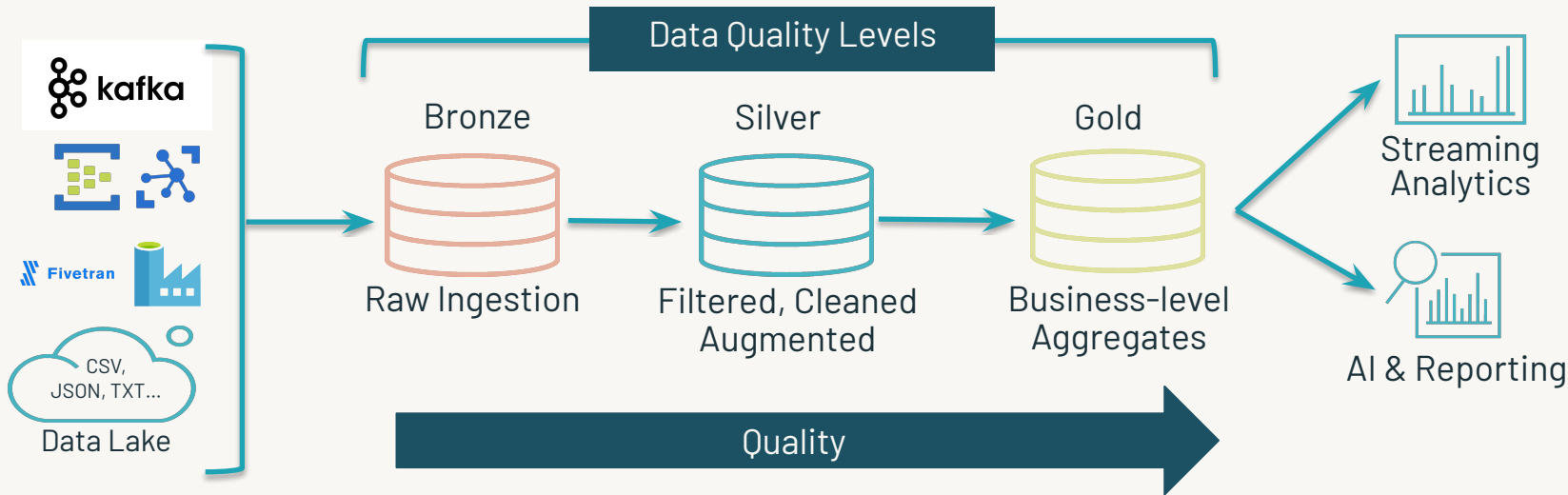


Open Data Lake

All Raw Data
(Logs, Texts, Audio, Video, Images)

Dimensional Modeling

Lakehouse Medallion Architecture



Delta Lake allows you to *incrementally* improve the quality of your data until it is ready for consumption.

Lakehouse Medallion Zones

(General Guidelines – each customer makes this their own)

BRONZE	SILVER	GOLD
Replica of Source for landing, archiving, re-processing, and Lineage Purposes	More normalized (3NF-like) / ODS / Data Vault-like / Write-optimized	Integration Solution Layer Star schema/ Kimball / Read-Optimized
Source System 1	Organized by Key Data Domains Like Customer, Product, Sales.	Projects/Use-cases (like C360, Marketing Analytics, Demand Forecasting etc.
Source System 2	Different Projects can bring their own additional data that ties to the Master Data Domains	Departmental Analytics Sandboxes
Source System N..	Master, X-REF Data, Replicated existing Data Marts	Data Science Sandboxes Feature Stores

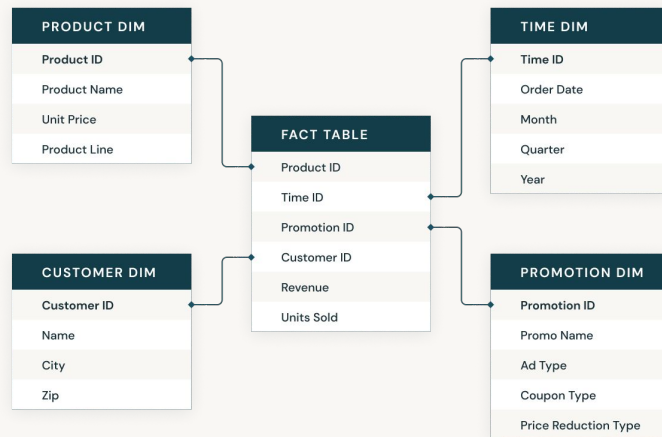


Dimensional Modeling

Kimball Approach

- Denormalize dimension and fact tables
- Use conformed dimensions
- Create and use surrogate keys for joins
- Use slowly changing dimensions as necessary
- Aggregate tables for scalability using DLT (Materialized Views) or in BI semantic layer tools

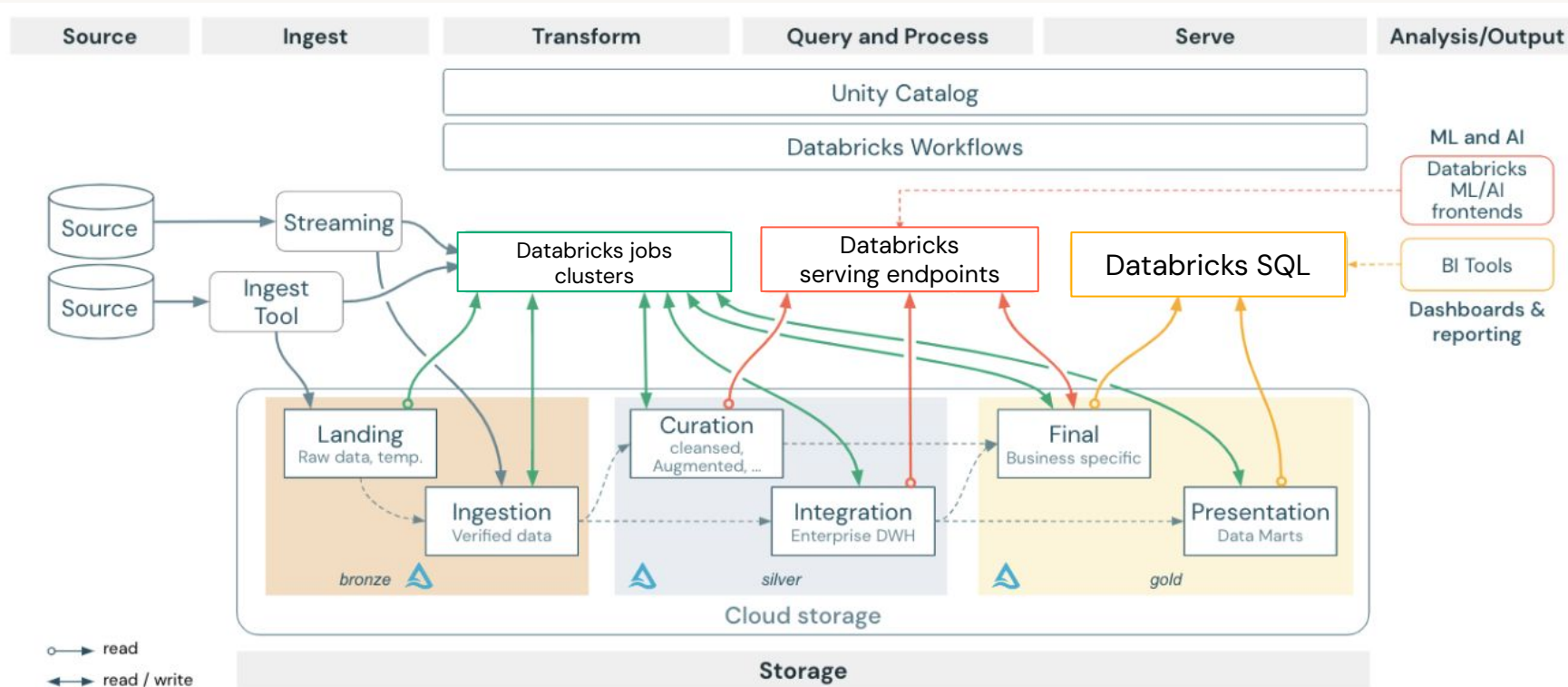
Star schema



BUSINESS PROCESSES	SHARED DIMENSIONS									
	Date	Customer	Product	Vendor	Promotion	Reseller	Sales Territory	Employee	Account	Organisation
Internet Sales	✓	✓	✓	✓	✓	✓	✓			
Reseller Sales	✓		✓		✓	✓	✓	✓		
General Ledger	✓								✓	✓
Sales Plan	✓		✓				✓			
Inventory	✓		✓	✓					✓	
Customer Surveys	✓	✓								
Customer Service Calls	✓	✓	✓					✓		



Process Diagram



Dimensional Modeling + Performance actions

0. **Kimball/Inmon** – use dimensional modeling industry principals
1. **Use Delta Tables & Databricks SQL (Photon)** – use Delta for your fact and dimension tables. Use DB SQL (Photon) for BI workloads
2. **Optimize your delta tables**
3. **Optimize your databricks warehouse**
4. **Optimize for your analytics tool (aka BI)**



Performance actions

—

Performance-Delta Tables

- Use Delta format
- Set NOT NULL where possible
- Use Predictive Optimization
- Use AUTO OPTIMIZE (if not using Predictive Optimization)
- Run ANALYZE TABLE regularly (if not using Predictive Optimization)
- Use Liquid Clustering
- Avoid using wide data types to reduce model size of Power BI Dataset



Performance–Databricks SQL



databricks

- Use SQL Warehouse instead of All-purpose compute
- Use SQL Serverless
- Scale out for concurrency. Consider auto-scaling
- Scale up for larger datasets. Start with Medium for most BI workloads
- Set Auto stop
- Utilize the query history in the UI and the System table
- Pushdown calculations to Databricks SQL
- Can use multiple SQL Warehouses for isolation of workload or BU
- Use Materialized Views
- Use Lakehouse Federation

Performance–Power BI

- Think Import vs Direct Query
- Star Schema
- Composite Model/Aggregations/Hybrid Tables
- Referential Integrity
- Query Folding/Push down transformation logic
- Use Efficient DAX calculations
- Evaluation configuration settings
 - Maximum connections per data source
 - Maximum number of simultaneous evaluations
 - Maximum number of concurrent jobs
 - MaxParallismPerQuery
- Reduce number of visuals on a page
- Apply/Clear All Slicers button
- Query reduction setting



Support Patterns

SCD Type 2 Pattern

Slowly Changing Dimensions Type 2+

Code Patterns for Batch

- Works using SQL, Python, and Scala
- Can use Merge or use separate operations to update and insert rows
- Hash of “slowly changing” attributes can provide an easy approach for comparing values

```
MERGE INTO customers
USING (
  -- These rows will either UPDATE the current addresses of existing customers or INSERT the new add
  SELECT updates.customerId as mergeKey, updates.*
  FROM updates

  UNION ALL

  -- These rows will INSERT new addresses of existing customers
  -- Setting the mergeKey to NULL forces these rows to NOT MATCH and be INSERTed.
  SELECT NULL as mergeKey, updates.*
  FROM updates JOIN customers
  ON updates.customerid = customers.customerid
  WHERE customers.current = true AND updates.address <> customers.address
) staged_updates
ON customers.customerid = mergeKey
WHEN MATCHED AND customers.current = true AND customers.address <> staged_updates.address THEN
  UPDATE SET current = false, endDate = staged_updates.effectiveDate -- Set current to false and er
WHEN NOT MATCHED THEN
  INSERT(customerid, address, current, effectiveDate, endDate)
  VALUES(staged_updates.customerid, staged_updates.address, true, staged_updates.effectiveDate, null)
```



Slowly Changing Dimensions Type 2+

Code Patterns for Streaming

- foreachBatch allows streaming pipelines to execute MERGE commands

```
# Function to upsert `microBatchOutputDF` into Delta table using MERGE
def upsertToDelta(microBatchOutputDF, batchId):
    # Set the dataframe to view name
    microBatchOutputDF.createOrReplaceTempView("updates")

    # =====
    # Supported in DBR 5.5 and above
    # =====

    # Use the view name to apply MERGE
    # NOTE: You have to use the SparkSession that has been used to define the `updates` dataframe
    microBatchOutputDF._jdf.sparkSession().sql("""
        MERGE INTO aggregates t
        USING updates s
        ON s.key = t.key
        WHEN MATCHED THEN UPDATE SET *
        WHEN NOT MATCHED THEN INSERT *
    """)
```

```
# Start the query to continuously upsert into aggregates tables in update mode
aggregatesDF.writeStream \
    .format("delta") \
    .foreachBatch(upsertToDelta) \
    .outputMode("update") \
    .start() \
```



Identity columns / Generate SK's in code

—

Generating Identity Columns

Old options for generating identity columns (surrogate keys)

monotonically_increasing_id – unique for the process, not for the table.

row_number() Rank OVER – requires a full table scan, shuffle, and sort.

zipWithIndex() – RDD function that is serialized in the driver

zipWithUniqueIndex() – an RDD function that runs in parallel

Row Hash with hash() – hash only the natural key

Row Hash with md5() – hash only the natural key

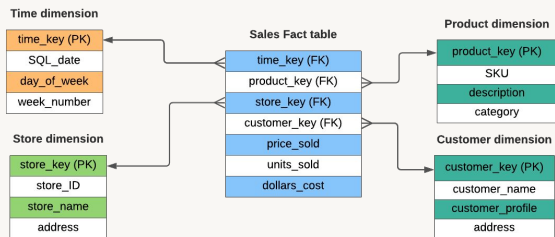
All of the old techniques had their own challenges and considerations and all are suboptimal!



Support for Identity Columns, Primary + Foreign Key Constraints

IDENTITY COLUMNS

- Define **IDENTITY** column on a table
- Delta can **automatically** generate **unique integer values** when new rows are added to the table with IDENTITY columns
- Users can also explicitly insert values for the IDENTITY columns



PRIMARY + FOREIGN KEY DECLARATIONS

- Declare unenforced Primary and Foreign keys with **ALTER TABLE**
- Visible in **INFORMATION_SCHEMA** and DESCRIBE TABLE
- Allow end users to understand **relationships** between tables

GOAL: Enable **data quality** and **easy table relationship discovery** for tools and users that are not familiar with the data model.



Generating Identity Columns

Delta Lake Identity Support Starting in DBR 10.4+

```
CREATE TABLE IF NOT EXISTS dim_loan
(
  Loan_sk BIGINT GENERATED ALWAYS AS IDENTITY,
  Loan_id BIGINT,
  .....
)
USING DELTA
LOCATION 'abfs://<container>@<storage account>'
```

Options

```
ALWAYS | BY DEFAULT
START WITH start
INCREMENT BY step
```

- Generated columns only work when pre-creating table
- Always option doesn't allow column override
- By Default option does allow column override but *doesn't enforce duplicates*
- Start With option allows you to start anywhere
- Increment option allows you to set the increment
- Modify settings with ALTER



Dimension Table Modeling

Designing your Dimension tables on Delta

```
CREATE TABLE IF NOT EXISTS dim_loan
(
  Loan_sk BIGINT GENERATED ALWAYS AS IDENTITY, --Identity Columns Functionality
  Loan_id BIGINT,
  Funded_amnt INT NOT NULL,
  Paid_amnt DOUBLE,
  Addr_state VARCHAR(100), --Support native database string types
  Type STRING,
  Load_date TIMESTAMP NOT NULL,
  LoanYear INT GENERATED ALWAYS AS ( YEAR(Load_date) ),
  LoanMonth INT GENERATED ALWAYS AS ( MONTH(Load_date) )
)
USING DELTA
PARTITIONED BY (LoanYear)
LOCATION 'abfss://<container>@<storage account>/'
TBLPROPERTIES (
  'delta.autoOptimize.optimizeWrite' = 'true',
  'delta.enableChangeDataFeed' = 'true');
ALTER TABLE dim_loans ADD CONSTRAINT dateWithinRange CHECK (timestamp > '1900-01-01');
```

- Use Delta
- Set Table Properties
- Set Location
- Generate SKs
- Can use more native SQL data types like varchar()
- Create constraints as needed
 - Null and Not Null
 - Add Constraint
- Partition as needed
- Optimize and Analyze

